Designing, decomposing, and delivering systemic software

Source Code Foundations

SEAtS | 892971

University of Southampton

Intended Learning Outcomes

By the end you will be able to:

1. Diagnose how requirements, architecture, and socio-technical constraints co-evolve across hardware, gateway, cloud, and UX components.
2. Apply Krischer et al.'s informed decomposition to partition systems into cohesive modules, interfaces, and protocols that can be optimised independently.
3. Use Smith & Kalantidou's systemic design mindsets (holism, leverage points, feedback loops) to maintain traceability from requirements → code → tests → deployment.
4. Evaluate readability, standards, linting, and error-handling patterns that keep codebases auditable and ready for CI/CD gates.
5. Build requirement-to-test coverage maps for multi-component case studies, articulating how code acts as living design documentation.

© University of Southampton

Agenda

1. Module framing: requirements, systemic leverage points, and source code as living design documentation.
2. Languages & abstraction stack—how code choices shape integration and traceability.
3. Informed decomposition (Krischer): coupling/cohesion, interface contracts, error models.
4. Readability, style standards, linting, and error/performance trade-offs linked to CI expectations.
5. Dependencies, version control workflows, CI/CD gates, and multi-layer testing as

© University of Southampton

Source Code as Living Design Document

Source code is the “blueprint + assembly manual” combined—every change should update both the map and the build instructions.

Code acts as the executable specification—requirements, architecture, and deployment assumptions converge inside each module.

Treat repositories like living system maps: docstrings, READMEs, and ADRs narrate stakeholder intent and boundary decisions.

Traceability tags (requirement IDs, test IDs) keep the living document aligned with audits, CI gates, and regulatory expectations.

© University of Southampton

Systemic Design Mindsets

Holism: When refactoring a module, consider upstream requirements, downstream data pipelines, and socio-technical impacts.

Feedback loops: Monitoring dashboards act as feedback channels; connect alerts back to backlog items for rapid adjustments.

Giga-maps & causality diagrams: Visualise requirements ↔ components ↔ test links so architectural shifts stay transparent.

Note: Causality diagrams show reinforcing or balancing loops; each arrow represents how one subsystem amplifies or dampens another.

© University of Southampton

From Idea to Source Code – System Journey



- Embed “traceability breadcrumbs” at each arrow (user stories, architecture decisions, code comments, test IDs).
- Innovation labs mirror CI/CD loops—each build collects stakeholder feedback before feeding the next concept.

© University of Southampton

Informed Decomposition

Structural metrics: Treat excessive coupling like a stiff mechanical joint; the more resistance between modules, the harder it is to evolve safely.

Purposeful slicing: Define component boundaries so teams optimise independently (similar to microservices or subsystems) while staying aligned to system objectives.

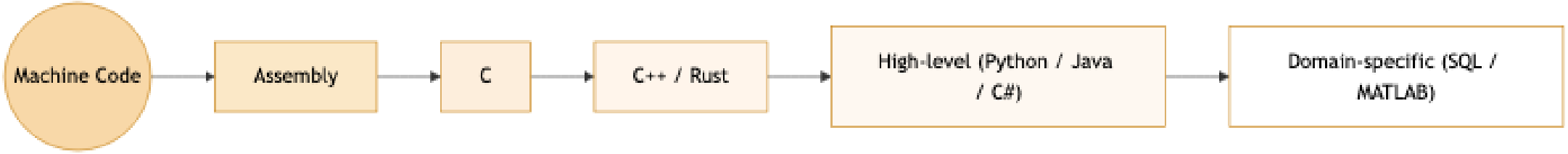
Interface discipline: Independent optimisation depends on explicit API contracts, shared schemas, and transparent assumptions.

Risk: Poor decomposition creates cascading failures—e.g., a firmware change silently breaking cloud analytics because interface expectations lived only in developer heads.

© University of Southampton

Programming Languages & Building Blocks

Layer	Typical Language	Role in Requirements Traceability	Notes
Firmware	C/C++	Deterministic timing for sensor specs	Constraints: memory, energy
Gateway	C++/Rust/Python	Protocol translation, buffering	Needs configurable retries & validation
Cloud services	Python/Node/Go	Analytics, APIs, ML flows	Emphasise CI, scaling, observability
Frontend	TypeScript	Visualization, stakeholder feedback loops	Accessibility & user telemetry



Each language is a specialized dialect in a multi-lingual negotiation—translation layers (APIs) prevent miscommunication.

© University of Southampton

Building Blocks Refresher + Practice

- **Variables & state:** Trace how state flows from sensor registers to UI state stores.
- **Control flow & exceptions:** Map where requirements demand deterministic decisions vs. recoverable errors.
- **Modules & packages:** Align each module with a requirement cluster (e.g., “REQ-3: secure data path”).

```
module SensorGateway
    function buffer(sample)
        assert sample conforms SchemaV2
        queue.enqueue(sample, retry_policy = EXPONENTIAL)
    end
end
```


© University of Southampton

Project Structure & Traceability Hooks

```
project/
  ├─ specs/requirements/REQ-*.md  ← stakeholder + non-functional context
  ├─ docs/gigamaps/               ← systemic diagrams, causality notes
  ├─ src/                         ← implementation modules (tagged w/ REQ IDs)
  ├─ interfaces/                  ← API schemas, protocol definitions
  ├─ tests/unit|integration|hil/  ← verification layers
  ├─ ci/                          ← pipelines, quality gates
  └─ ops/                         ← deployment, monitoring scripts
```


© University of Southampton

Readability vs Ambiguity

```
# Ambiguous naming + magic numbers
if status == 2 then send(x)
```

```
# Intent-rich alternative
if sensorPacket.isValidChecksum() then
    telemetryChannel.transmit(sensorPacket.withTraceId(traceId))
end
```


© University of Southampton

- Meaningful names encode design decisions and reduce onboarding time.
- Comments explain *why* (link to requirement or risk mitigation) not *what*.
- Cross-reference tests: `// Verified by tests/integration/gateway_checksum.spec`.

Adopt published standards (e.g., Google Style Guides, PEP 8, MISRA C) that balance readability, tooling compatibility, and domain constraints.

Linting (clang-tidy, pylint, eslint) enforces these conventions automatically, catching deviations and reinforcing collective discipline.

Google emphasizes readability and consistency across teams; PEP 8 prioritizes Pythonic idioms; MISRA enforces strict safety rules for embedded systems.

© University of Southampton

Compare standards

```python
# Google: explicit docstring + type hints
def compute_area(radius: float) -> float:
    """Return area of a circle."""
    if radius < 0:
        raise ValueError("radius must be non-negative")
    return math.pi * radius * radius
```

```python
# PEP 8: snake_case, minimal comments
def compute_area(radius):
    if radius < 0:
        raise ValueError("radius must be non-negative")
    return math.pi * radius * radius
```

```c
/* MISRA C style */
float compute_area(const float radius)
{
    if (radius < 0.0F)
    {
        report_error(RADIUS_NEGATIVE);
        return 0.0F;
    }
    return (float)(PI * radius * radius);
}
```


© University of Southampton

Coupling vs Cohesion

Cohesion: Keep modules focused on a single responsibility so changes remain localized and intent stays obvious.

Coupling: The tighter the links between modules, the more resistance you create to change—treat it like a stiff joint that slows every motion.

Target state: High cohesion + low coupling emerges from clear boundaries, dependency inversion, and well-documented API contracts.

© University of Southampton

Interface Contracts & Protocol Design

Define the intent: Capture payload schemas, timing guarantees, and error codes as you would clauses in a legal agreement.

Version discipline: Maintain compatibility matrices (e.g., “cloud v3 requires firmware ≥1.4”) so change impact is obvious.

Testing hooks: Contract tests in CI prevent silent drift and prove each interface still satisfies stakeholder acceptance criteria.

© University of Southampton

Error Handling Models

Fail-fast: Abort on unrecoverable faults (e.g., cryptographic mismatch) so safety and security requirements never degrade silently.

Fail-safe / degraded: Gateways buffer locally when the cloud is unreachable and raise alerts to keep stakeholders informed.

Compensation: Saga-style workflows roll back partial actions; thorough logging keeps traceability intact.

Risk: Silent retry storms can overload dependencies—use circuit breakers and metrics tied to non-functional requirements.

© University of Southampton

```
# Good: explicit failure path
if sensorPacket.hasValidSignature() then
    telemetryChannel.transmit(sensorPacket)
else
    raise SecurityException("Invalid signature", packetId)
    audit.logIncident(packetId, "signature_mismatch")
end
```

Good error handling surfaces the fault, halts unsafe behaviour, and records context for traceability.

```
# Bad: silent retry loop
try
    transmit(sensorPacket)
catch Exception
    retry()  # no backoff, no logging
end
```

Bad handling swallows the exception, retries blindly, and hides evidence needed for audits or recovery.

© University of Southampton

Performance vs Readability Trade-offs

Context matters: Firmware often prioritises deterministic timing, while cloud services can emphasise readability and observability.

Evidence first: Profile before sacrificing clarity, and document decisions in ADRs linked to commits.

Benchmark discipline: Compare naive vs optimised implementations and capture CPU/memory plus maintainability notes.

© University of Southampton

Design Patterns for Modularity

Ports & Adapters: Separate domain logic from infrastructure so systemic concerns stay isolated.

Observer/Event streams: Model feedback loops and integrate monitoring hooks directly into the architecture.

State machines: Capture firmware or gateway behaviour with explicit transitions tied to requirements.

© University of Southampton

Dependencies & Integration Strategies

Version hygiene: Pin dependencies and document compatibility so everyone knows where friction may occur.

Governance: Add dependency review steps in CI to surface transitive vulnerabilities or licensing red flags.

Integration layers: Use API gateways/adapters to decouple hardware timing constraints from cloud scaling needs.

Failure mode: Shared libraries updated without gateway rebuilds; mitigate via automated integration tests + release gates.

© University of Southampton

Version Control Workflows

Git-Flow: Feature + release branches suit hardware-in-the-loop sign-offs and scheduled drops.

Trunk-based: Short-lived branches and feature flags enable continuous delivery if automation is strong.

Traceability links: Tie commits to requirement IDs and leverage points so socio-technical effects stay visible.

© University of Southampton

CI/CD Gates & Systemic Innovation Labs

Safe-to-fail loops: Treat CI pipelines as innovation labs where teams iterate with rapid feedback.

Gate sequence: Linting → unit tests → integration tests → hardware-in-the-loop → staging deploy keeps quality layered.

Socio-technical checks: Embed accessibility scans, security reviews, and stakeholder sign-offs alongside technical gates.

© University of Southampton

Testing Layers – Code to Hardware

Layer	Purpose	Tooling	Link to Requirements
Unit	Validate functions/modules	pytest, Unity, Jest	Functional requirements
Integration	Verify interactions/interfaces	Postman, Robot, custom harness	Interface contracts, schema versions
System/End-to-end	Validate workflows & UX	Cypress, Behave	Stakeholder scenarios
Hardware-in-the-loop (HIL)	Firmware ↔️ physical components	Lab rigs, simulators	Safety, timing, energy budgets
Continuous monitoring	Production feedback loops	APM, observability stacks	Non-functional requirements, leverage points

© University of Southampton

Quality Attributes & Trade-offs

Clarity + Correctness: Use code reviews and assertions tied to requirements to prove behaviour.

Efficiency + Security: Track latency, CPU, and threat models, and record the rationale for optimisations.

Reusability + Resilience: Feature flags, retries, and circuit breakers protect both modularity and uptime.

© University of Southampton

Systemic Leverage Points (Meadows) for Software

Parameters: Config defaults, retry counts, and thresholds are easy to tweak but have shallow impact.

Feedback loops: Observability dashboards or automated alerts reshape behaviour far faster than manual reviews.

Paradigms: Cultural shifts like “documentation-as-code” or “test-first architecture” produce deep, lasting change.

© University of Southampton

Case Study – Distributed Sensing Platform

- **Scenario:** Smart campus air-quality network with firmware sensors, gateway, cloud analytics, and web dashboard.
- **Requirements:** R1 data freshness < 60s, R2 accuracy ±2%, R3 secure transport, R4 stakeholder dashboards, R5 audit trails.
- **Traceability:** Every component references R1–R5 in code comments, tests, and CI artifacts.

Add diagram: four blocks (Sensor → Gateway → Cloud API → Dashboard) with arrows annotated by protocols (SPI, MQTT, HTTPS, GraphQL) and test hooks.

© University of Southampton

Case Study – Component Breakdown

Component	Interfaces	Source Code Focus	Testing & CI Hooks	Risks/Fails
Sensor firmware	SPI to sensors, BLE to gateway	ISR timing, calibration tables, power mgmt	Unit tests via simulators, HIL bench	Drift, battery depletion, timing jitter
Gateway	BLE ↔ MQTT, local storage	Buffer mgmt, retries, encryption	Integration tests w/ emulated sensors, chaos tests	Queue overflow, stale certs
Cloud analytics	MQTT ingest, REST, data lake	Stream processing, ML calibration, API versioning	CI contract tests, load tests, blue/green deploy	Schema drift, scaling bottlenecks
Web dashboard	GraphQL/REST	Data viz, alert workflows, role-based access	End-to-end tests, accessibility scans	UX regressions, PII leaks

Highlight decomposition challenges: firmware updates require validating gateway compatibility; treat like mechanical stiffness.

© University of Southampton

Case Study – Interface Contracts & Traceability

- API schemas stored in `interfaces/air-quality/v2.proto` with change logs referencing requirements.
- Contract tests run nightly to ensure gateway payloads still align with cloud expectations.
- CI dashboards show requirement coverage: R1 (latency < 60s) tests, R2 (accuracy ±2%) calibration checks, R3 (secure transport) security scans.

© University of Southampton

Case Study – Coordination & Project Management

- **Roles:** Firmware squad, gateway/API squad, analytics squad, UX squad; shared system architect ensures coherence.
- **Coordination ceremonies:** Interface reviews, shared backlog board, incident post-mortems feeding requirements updates.
- **Risk management:** Feature toggles and staged rollouts mitigate systemic shocks.

© University of Southampton

Summary

- Source code is a living design document—holistic maps, causality diagrams, and requirement IDs keep socio-technical intent explicit.
- Informed decomposition + systemic mindsets ensure cohesive modules, resilient interfaces, and visible leverage points.
- Readability standards, linting, and disciplined error handling underpin CI/CD gates, quality metrics, and regulatory evidence.
- Traceability threads (requirements → code → tests → monitoring) plus rigorous coordination make multi-component deployments auditable and adaptable.

© University of Southampton